

接受之后：基于 eBPF 校准的残差语言与怪异机器论断图谱

独立研究者·emtanling@gmail.com

Chengao Zhang

版本说明 (2026-07-20): 本文件是供作者核对论证的中文阅读版, 不是逐句翻译, 也不是投稿规范源。英文 PAPER_REPORT.tex 是当前公开技术报告源; 当前没有投稿规范 PDF, 未来投稿必须另行生成匿名构建。如中文表述与英文公开技术报告源不一致, 以英文为准。特别地, Stock Linux V1 的旧适配器结论已被证据模型 v1 更正: 它不是实际 Linux 目标 M_{Linux} 的 R 结果; 旧适配器的构造模型记为 $M_K^{\text{legacy}} = M_{\text{adapter}}$ 。新增的 Stock-R V2 只对自身的 exact operational-prune query 给出 proof-bound NONFACTORING, 不回填 V1, 也不构成一般 Linux 功能报告或漏洞结论。

摘要

语言理论安全把输入边界建模为形式语言识别器, 把下游处理建模为解释。在程序验证器边界, 验证器是边界制品的外层识别器; 已接纳制品又可解释数据并诱导运行时操作词, 而任何已计算报告都是另行明确声明的抽象。边界制品不仅包括代码, 也包括元数据、配置、证明对象与体系结构控制结构; 被诱导的操作不必是由制品自身执行的 CPU 指令。本文提出五节点论断图谱, 严格区分制品接受 (A)、同后缀因果状态区分 (C)、有界可编程性 (P)、由输出见证的已计算报告不可因子分解性 (R) 以及策略/威胁义务 (W)。未来观测商与行为因子分解判据区分以已接受制品为索引的因果语言和相对于报告实例的残差语言。该判据只在选定上下文纤维上的**唯一单元报告映射**成立时适用; 本文不声称每个安全健全但不完备的验证器都必然承载怪异机器。

本文给出统一可控、可观察、可重置门以及固定已接受解释器的证明义务。在精确调度、可接纳性、串行化、源码到目标对应与帧保持前提下, 前缀归纳给出有界组合结果。在 Linux eBPF 校准案例中, 在所声明的映射更新、重置和无干扰契约下, 一个专用、预分配、非 LRU、容量为 2 的哈希映射充当 NAND 门: 重置后保留一个哨兵项, 输入比特选择更新已有键或插入新键, 第二次更新的成功谓词作为输出。一个固定已接受程序解释输入至多 64 个、门至多 512 个、活跃规范导线至多 578 条的 NAND DAG。

源码快照化的 Linux/aarch64 运行覆盖具名与随机电路、联合边界、串行复用、机制对照和畸形描述符。作者另行运行的语义审计器重建描述符与电路语义, 自行签发的清单检查证据包完整性。这个解释器载体记录 A; 在声明的具体映射服务与无干扰契约下给出条件性 C 见证; 在更多实现前提下支持 P, 但不建立 R 或 W。固定辅助可执行实例针对自身计算出的报告建立 $R(M_{\text{linux}_r\text{-aux}_v1})$ 。冻结的 Stock Linux V1 实验只记录了一条 exact-level-0 的 states_equal/is_state_visited operational-prune 边, 以及同一后缀下两个已捕获运行的 MAY 差异; 旧适配器在构造模型中失败因子分解, 但不能转写为实际 Linux 目标 M_{Linux} 的 R 证明。按 evidence-model v1, V1 的 outcome eligibility 为 NOT_ESTABLISHED, 所以 exact operational-prune query 为 UNKNOWN。新的 Stock-R V2 是独立的前瞻实验: runner 先封存对象、翻译后字节码、BTF、内核、源码闭包、检查器和运行时身份, 再写出 proof/must-outcome-proof.json 与 proof/history-case-binding.json, 把选中的两条剪枝历史绑定到证明案例 0/1、同一前沿、报告单元与后缀。一次新的 stock Ubuntu 6.17.0-35-generic 运行给出 outcome_eligibility.status = ESTABLISHED, method = MUST_OUTCOME_PROOF_WITH_HISTORY_CASE_BINDING, assessment.status = NONFACTORING, assessment.scope = EXACT_STOCK_R_V2_QUERY。通用证据图/证明 DAG 检查器随后把 V1 判为 BLOCKED/INCONCLUSIVE、把 V2 判为精确范围的 CERTIFIED/NONFACTORING; 十二种敌

对变异中，五种请求层越权主张被阻断，七种证明级或完整性/依赖攻击被拒绝。受保护的 Contextual Residual Lifting (CRL) 扩展加入 verifier-contract residuality theorem 和可检查的 DERIVED_CONTEXTUAL 运输链；在两个翻译后字节码 digest 均不同于 V2 源程序且彼此不同的 VM 目标上，检查器导出独立的 exact AT(target)/TRANSPORTED 证书，且上下文敌对矩阵拒绝范围、完整性、选择、派生链和循环性攻击。各载体不可合并：辅助 R、Stock-R V2 与 contextual targets 均不与解释器 P 证据链接，也没有任何结果建立 W 或怪异机器。

分类上，C 是共同最小条件，P 与 W 是彼此独立的细化：行为主体可控制且非预期的 C 见证构成涌现怪异机器；加入 P 得到可编程怪异机器，加入与同一 C 链接的策略排除效果则得到策略活跃怪异机器；更窄的报告形状类位于两分支交汇处。本文对代码复用、数据导向、元数据/证明模型、页故障、解析器、跨语言、跨层、嵌入式中断、推测/微体系结构和分布式案例作结构化回溯，以说明为什么需要该分类格。这些回溯只转述原论文建立的成分，不是定义 2 证书，也不改变 eBPF 证据边界。

关键词： 语言理论安全，识别器，残差语言，怪异机器，程序验证，抽象解释，eBPF。

1. 引言

语言理论安全把输入边界视为语言识别边界：下游处理是解释，输入提供被解释的程序 [1]–[4]。本文把这一识别器形状用于程序验证。问题从“接纳之后”开始：边界接纳代码、元数据、配置、证明对象或体系结构控制结构，制品随后诱导有状态操作语言。若要把这类下游行为称为残差的、可编程的、由形状诱导的或怪异机器，分别需要什么证据？在这一分层视角下，接受、解释和报告抽象分别回答 L_V 中的成员关系、 I 中的行为以及 Report_V 如何分组这三个不同问题；eBPF 案例把一般制品特化为已接受程序。

制品语言和接受后的操作语言具有不同载体。验证器可以认证有界执行、辅助函数类型使用和内存访问纪律，而不必认证通过每个已文档化运行时服务实现的完整关系。已接受程序可以选择操作、保留状态、观察返回、重置和组合效果；这些事实本身既不证明验证器缺陷，也不证明报告遗漏。

载体	A/C/P/R/W 状态	证据解释
wm_circuit 解释器	A 已记录；C 在声明的服务/无干扰契约下有条件成立；P 在源码/对象、串行化与帧前提下得到支持；R、W 未建立	构造性论证与经审计回归证据；解释器前沿没有已计算单元提取器
$M_{linux_r_aux_v1}$	A、C、R 在有限自定义元组内成立；P、W 未建立	可执行有限模型证书；R 仅针对其计算出的自定义报告，不声称与 stock Linux 存在 refinement 或 bisimulation
Stock Linux V1 ($M_{Linux}, \text{rac_single}$)	A 针对冻结对象/内核元组已记录；一个 operational-prune 边和两个已捕获运行的 MAY 差异已记录；实际 Linux 的 exact operational-prune 结论为 UNKNOWN；P、R、W 未建立	旧适配器 $M_K^{\text{legacy}} = M_{\text{adapter}}$ 的二状态/单动作构造模型出现因子分解失败，但不能转写为 M_{Linux} 的 R；功能报告不在本证据范围
Stock-R V2 ($M_{Linux}^{V2}, \text{rac_v2}$)	受控对象/内核元组的 A 已记录；operational prune、runtime replication、已检查 must-outcome proof 与 history-case binding 建立 exact-query NONFACTORING；P、W 未建立	针对 V2 array-map witness 的声明 operational-prune report 的 proof-and-binding 结果；没有 Linux 功能报告契约，也没有更广运行/元组结论

载体	A/C/P/R/W 状态	证据解释
Stock-R contextual target ($M_{Linux}^{V2.ctx}$, contextual)	两个生成式上下文目标的 A 已记录; CRL 从 exact V2 源证书导出 exact AT(target)/TRANSPORTED NONFACTORING; P、W 未建立	两条 DERIVED_CONTEXTUAL 证书链, 目标翻译后字节码 digest 均不同于 V2 且彼此不同; 没有 family、编译器正确性 或一般 Linux 结论

每一行都有独立的制品、执行载体、报告接口和证据类型。`wm_circuit` 行承载本文的 A/C/P 构造; 辅助行只针对自定义的报告生成识别器提供正面 R 结果; Stock Linux V1 的 M_{Linux} 行仅保留操作剪枝边与 MAY 差异的记录, 且旧适配器模型 $M_K^{legacy} = M_{adapter}$ 的结果不能成为 M_{Linux} 的 R 结论。V2 行则是另一个已接受对象和见证上的 proof-bound exact-query operational-prune 结果。contextual 行只给出由 V2 派生的 target-bound transport certificate, 不是新的源证明或 family theorem。不得跨行组合不同节点, 任何已评估行都没有建立 W。

节点 C 和 R 不可共用一个名称。以已接受制品为索引的同后缀语言记为 L_{causal} ; 只有实际计算单元在全部可接纳条件下发生碰撞, 词才进入 L_{res}^R 。P 与 R 在 C 后分叉。已发表案例还要求第二处分叉: 可编程性与策略效果也彼此分叉。本文把行为主体可控制且非预期的 C 见证记为 $WM_{emergent}$, 分别以 P 和链接的 W 细化为 WM_{prog} 与 WM_{policy} ; 更窄的“由契约形状诱导、相对于识别器”的类别位于两分支交汇处, 还要求 R、Doc、Conf 与 Gran。这样既不会把图灵完备演示自动变成 exploit 主张, 也不会排除没有通用基、但策略上有意义的弱机器。

本文贡献为: (1) 带类型的“识别器—解释器—报告”A/C/P/R/W 论断图谱; (2) 以 C 为底座、P/W 分叉的严格怪异机器分类格, 以及九组已发表案例族的证据受限回溯; (3) 包含 E1–E4-D 的有界组合证明义务; (4) 一个以 $D_{64,512}$ 为域的 eBPF NAND 解释器校准案例; (5) 严格分载体的 R 证据, 以及 V1-blocked/V2-certified/DERIVED_CONTEXTUAL 的精确证书控制。固定辅助实例针对自定义报告检查有限报告单元碰撞及四个负对照。独立的 Stock Linux V1 捕获记录真实 exact-0 剪枝边和同一后缀下的两个已捕获结果; 旧适配器仅在构造的二状态/单动作模型中失败因子分解。按照 evidence-model v1, 它不建立真实 Linux 的 R、定义 2 实例或轨迹局部 R 证书: exact query 与更广问题均为 UNKNOWN, 功能报告为 OUT_OF_SCOPE。Stock-R V2 通过前瞻 runner、重复运行、已检查 must-outcome proof 和 history-case binding, 只对 EXACT_STOCK_R_V2_QUERY 建立 NONFACTORING; 通用检查器保持这一范围并拒绝十二种敌对变异。CRL 再从 exact V2 源证书派生两个独立 contextual target 证书, 且不消费目标终局 verdict。辅助 R、V2 与 contextual target 结果都不与解释器 P 证据链接。实证范围限于一个特权、离线 Linux/aarch64 内核构建族、四个承载主张的已接受 eBPF 制品、三个额外的已接受解释器控制变体以及一个由自定义识别器接受的辅助制品; 不是一般 Linux 报告不透明性定理、并发部署、无界机器、漏洞或策略级怪异机器。

2. 相对于识别器的残差语言

2.1 识别、执行与报告

令 V 为边界制品识别器, I 为具体执行语义, 状态载体为 Σ_I , 轨迹载体为 $\mathcal{T}_I$; I 可包含指令引擎、加载器、元数据解释器、协议状态、体系结构机制、有状态服务及相关环境。边界制品是其接纳会决定后续解释的代码、结构化数据、元数据、配置、证明对象或控制结构。本文仍用 P 表示一般制品; 在 eBPF 特化中它是程序。因此 $Tr_I(P) \subseteq \mathcal{T}_I$, 且

$$L_V = \{P \mid V(P) = \text{accept}\}.$$

对可选的允许轨迹集合 $\text{Safe} \subseteq \mathcal{T}_I$, 安全健全性是

$$\forall P \in L_V. \text{Tr}_I(P) \subseteq \text{Safe}.$$

安全健全性是前提，不能由一次成功加载推出；它也不同于抽象变换器的完备性 [5]。已接受语言、实际计算报告、传递健全性与完备性是不同判断。

若报告在讨论范围内，令

$$\text{Report}_V(P, \ell) \subseteq A_\ell, \quad \gamma_\ell : A_\ell \rightarrow \mathcal{P}(\Sigma_I)$$

分别为前沿 ℓ 实际计算的有限单元集及声明的具体化。覆盖要求

$$\text{Reach}_I(P, \ell) \subseteq \bigcup_{a^\# \in \text{Report}_V(P, \ell)} \gamma_\ell(a^\#).$$

只有同一个实际计算单元同时包含两个状态，二者才是“共同覆盖”；相似日志文本不足以证明单元身份、具体化或共同覆盖。

2.2 因果词与观测契约

对 $P \in L_V$ 和前沿 ℓ ，令 $\Sigma_{\text{op}}(P)$ 是边界相关运行时操作的有限字母表， $W_{\text{run}}^\ell(P) \subseteq \Sigma_{\text{op}}(P)^*$ 是通过声明的驱动接口、由呈现或配置 P 可从该前沿诱导的词。诱导包括直接执行但不以此为限：ELF 重定位表可驱动加载器，页表可驱动故障机制，路由配置可驱动分布式状态迁移。C 本身不包含行为主体控制；该条件在怪异机器分类时另行检查。节点 C 使用同后缀测试，不把所有普通运行轨迹重新命名为残差。

比较前固定带类型的观测契约

$$K_{\text{obs}} = (\rho_{\text{obs}}, \text{Obs}, \text{Slice}, \text{Env}),$$

其中 $\rho_{\text{obs}} : \Sigma_I \rightarrow R_{\text{obs}}$ 选择候选状态， $\text{Obs} : \mathcal{T}_I \rightarrow O_{\text{obs}}$ 投影完整轨迹， Slice 为每个词 w 指定上下文投影 $\text{ctx}_w : \Sigma_I \rightarrow C_w$ ， Env 是固定环境实例集合。上下文必须包含后缀或观察者读取的全部非选定分量；环境实例固定资源配置、调度、非确定性与外部干扰选择。

写作 $\llbracket w \rrbracket_{I,e}(\sigma) = (\tau, \sigma')$ 。契约在 $X \subseteq \Sigma_I$ 上对 w 健全，当任意 $e \in \text{Env}$ 和 $\sigma, \sigma' \in X$ 若满足

$$\rho_{\text{obs}}(\sigma) = \rho_{\text{obs}}(\sigma'), \quad \text{ctx}_w(\sigma) = \text{ctx}_w(\sigma'),$$

则两次执行有相同的有定义性；若均有定义，则 $\text{Obs}(\tau) = \text{Obs}(\tau')$ 。没有这一非干扰条件，不接纳 C 见证。

定义 1 (因果状态介导词族)。词 w 在 (P, ℓ) 处因果，当 $P \in L_V$ 、 $w \in W_{\text{run}}^\ell(P)$ 、 K_{obs} 在 $\text{Reach}_I(P, \ell)$ 上对 w 健全，并存在同一 $e \in \text{Env}$ 及 $\sigma_0, \sigma_1 \in \text{Reach}_I(P, \ell)$ ，使两次后缀执行有定义、终止且

$$\text{ctx}_w(\sigma_0) = \text{ctx}_w(\sigma_1), \quad \rho_{\text{obs}}(\sigma_0) \neq \rho_{\text{obs}}(\sigma_1), \quad \text{Obs}(\tau_0) \neq \text{Obs}(\tau_1).$$

定义依赖标签族

$$L_{\text{causal}}(V, I; K_{\text{obs}}) = \{(P, \ell, w) \mid w \text{ 在 } (P, \ell) \text{ 处因果}\}.$$

制品与前沿是类型标签。宿主描述符不是 L_V 的另一个元素，也不会自动成为运行时词；它先成为配置，由固定解释器诱导调度，只有同后缀见证进入 L_{causal} 。

2.3 行为商与报告相对残差

固定确定性规约

$$D = (X_D, S_D, A_D, O_D, \delta_D, \lambda_D, s_D, \text{Obs}_D),$$

其中 $X_D \subseteq \Sigma_I$ 是可接纳具体区域, S_D 是状态载体, A_D 是操作字母表, O_D 是输出字母表, $\text{Obs}_D : O_D^* \rightarrow O_{\text{obs}}$, 并且

$$\delta_D : S_D \times A_D \rightarrow S_D, \quad \lambda_D : S_D \times A_D \rightarrow O_D$$

是同定义域偏函数。对每个使用规约的 (P, ℓ) , 固定单射操作编码 $\iota_{P, \ell} : A_D \rightarrow \Sigma_{\text{op}}(P)$ 并同态扩展至词。

当且仅当对每个 $\sigma \in X_D$ 和 $a \in A_D$, 具体编码步骤存在恰好对应 $\delta_D(s_D(\sigma), a)$ 有定义, 并且每个 $\sigma \xrightarrow{\iota_{P, \ell}(a)/o} \sigma'$ 满足

$$\lambda_D(s_D(\sigma), a) = o, \quad s_D(\sigma') = \delta_D(s_D(\sigma), a), \quad \sigma' \in X_D,$$

称 $(s_D, \iota_{P, \ell})$ 在 X_D 上**操作充分**。因此编码是语义重命名, 不能让同一投影状态隐藏不同有定义性、完整输出或后继投影; 影响转移的环境选择必须被固定或纳入 S_D 。

令 $\text{Out}_D(r, w)$ 为迭代 (δ_D, λ_D) 得到的偏完整输出词, $\text{Def}_D(r, w)$ 表示其有定义, 并取 $\mathcal{W}_D = A_D^*$ 。定义

$$r \sim_D r' \iff \forall w \in \mathcal{W}_D. [\text{Def}_D(r, w) \iff \text{Def}_D(r', w)] \wedge [\text{Def}_D(r, w) \Rightarrow \text{Out}_D(r, w) = \text{Out}_D(r', w)].$$

$\sim_D$ 是右同余; 若 $Q_D = S_D / \sim_D$ 有限, 则 X_D 上执行诱导商状态上的偏 Mealy 转导器 [6]–[9]。本文只测试一个已接受制品、一个前沿上的实际计算单元, 不主张提出新的通用完备性理论。

对具体集合 F , 共同启用延续为

$$\mathcal{W}_D(F) = \{w \in \mathcal{W}_D \mid \forall \sigma \in F. \text{Out}_D(s_D(\sigma), w) \downarrow\}.$$

命题 1 (上下文纤维上的行为因子分解)。固定 $P, \ell, D, K_{\text{obs}}$ 和非空 $F \subseteq \text{Reach}_I(P, \ell) \cap X_D$ 。要求:

1. $\iota_{P, \ell}(\mathcal{W}_D(F)) \subseteq W_{\text{run}}^\ell(P)$;
2. 观测契约对全部编码共同延续健全;
3. F 上全部对应上下文投影相同;
4. 观察者兼容: 若编码词 w 从 σ 执行产生具体轨迹 τ , 则

$$\text{Obs}(\tau) = \text{Obs}_D(\text{Out}_D(s_D(\sigma), w));$$

5. **唯一单元条件**:

$$\forall \sigma \in F. \exists! a^\# \in \text{Report}_V(P, \ell). \sigma \in \gamma_\ell(a^\#).$$

令 $\pi_R : F \rightarrow \text{Report}_V(P, \ell)$ 把状态映射到其唯一实际计算单元, $\beta_D(\sigma) = [s_D(\sigma)]_D$, $F_{a^\#} = F \cap \gamma_\ell(a^\#)$ 。则

$$\exists h : \pi_R(F) \rightarrow Q_D. \beta_D = h \circ \pi_R$$

当且仅当

$$\forall a^\# \in \pi_R(F). |\beta_D(F_{a^\#})| \leq 1.$$

所以, 报告在 F 上不可因子分解, 当且仅当一个唯一实际计算单元包含来自两个不同 $\sim_D$ 类的状态。唯一单元条件是使 π_R 成为函数的必要前提; 若报告标签重叠, 必须另行声明成员签名映射, 不能沿用本命题。

记 $\text{Adm}(P, \ell, D, F; K_{\text{obs}})$ 表示接受、操作充分、可达非空纤维、运行词包含、观测者兼容、健全观测契约、共同上下文和唯一单元条件全部成立。唯一单元条件使非空的 $F_{a^\#}$ 构成不交覆盖; 单元重叠的报告需要另行声明成员签名映射, 不属于本判据。

可接纳性是内部语义定型条件, 并不自动排除事后选择。每个正面实例必须声明 D 、 F 、观测者、后缀和报告接口究竟是在看到区分性执行之前预先确定, 还是在结果已知后事后选定。若要提出前瞻性或报告层面的一般主张, 还须提供独立于见证输出的选择来源, 例如预先登记的协议或外部规定的报告契约。事后元组可以严格满足定义 2, 但仅凭这一事实不能刻画识别器意图中的报告、更广的执行域或一类实现。

定义 2 (相对于报告的输出见证残差语言)。 对一个可接纳元组, 带标签词 $(P, \ell, D, F, a^\#, w)$ 属于输出见证的报告相对残差, 当: 编码词属于 L_{causal} ; $w \in \mathcal{W}_D(F)$; 定义 1 的两个见证位于 F ; 二者唯一报告标签同为 $a^\#$; 且其 β_D 类不同。 L_{res}^R 是所有可接纳元组上这类带标签词的并集。

该定义只是报告因子分解失败的**输出见证子集**。商关系也区分有定义性, 因此可能存在不可因子分解, 却没有一个两边都终止并产生不同输出的词。保留的 `wm_circuit` Linux 日志不提供实际计算单元提取器、具体化或唯一单元映射; 所以该解释器载体只给出条件性 L_{causal} 见证, 不建立 L_{res}^R 。第 5.7 节区分 `rac_single` V1 的 fail-closed 复核和 `rac_v2` 的 proof-bound exact-query 结果。

2.4 形状、缺陷与策略

缺陷诱导缝隙违反声明的识别器、报告或运行时契约。一个使用已文档化且安全保持语义、并在报告意图认证的关系上具有 L_{res}^R 见证的案例, 只是契约形状缝隙的证据; 仍须以 `Conf` 与 `Gran` 排除错误变换器、汇合或提取器。“非预期”相对于具名意图契约, 而不等同于“未文档化”或“漏洞”: 若已文档化机制组合成名义计算模型之外的解释, 仍可满足 `Unint`。第 7 节把行为主体可控制的非预期因果机器与其可编程、策略活跃细化分开。普通且符合意图的有状态 API 即使实现转导器, 也不满足怪异机器分类。

3. 有界状态介导的电路实现

3.1 E1–E3: 从区分到可复用门

固定规约 D 、规范重置商类 $q_0 \in Q_D$ 及非空 $\text{Adm}_G \subseteq X_D$ 。门基 $(P_G, \text{reset}, G, \text{observe}, D, \text{Adm}_G)$ 的操作编码在 X_D 上操作充分, 并满足:

- **E1 (因果基与观测):** 存在输入 x_0, x_1 、前缀 p_0, p_1 、同一剩余后缀 w 和内部前沿 ℓ_G , 使 $u_G(x_i) = p_i w$; 两个前缀所述状态是定义 1 对编码 w 的见证; 完整门读出恰等于该因果观测, 并正是 E4-D 写入的门结果。
- **E2 (统一输入控制):** 同一 P_G 中的分派器读取 $x \in \{0, 1\}^2$ 。从 q_0 中任意可接纳状态执行 $u_G(x)$ 都有定义, 且

$$\text{observe}(\text{Out}_D(s_D(\sigma), u_G(x))) = g(x).$$

- **E3 (重置):** 重置词对所有可接纳状态有定义, 保持导线单元并返回 q_0 ; 固定上下文和导线相同的两个重置结果, 对所有输入产生相同完整输出词。

E1 防止读出与状态区分无关; E2 防止实验者从外部提供真值表; E3 防止通道只能使用一次。

3.2 描述符域与 E4-D

域 $D_{64,512}$ 的描述符为

$$d = (m, n, (s_i^0, s_i^1)_{0 \leq i < n}),$$

其中 $0 \leq m \leq 64$ 、 $0 \leq n \leq 512$ 且 $0 \leq s_i^b < 2 + m + i$ 。导线 0、1 为常量，输入位于 2 至 $2 + m - 1$ ，门 i 写规范目的 $2 + m + i$ ；最多 578 条活跃规范导线，最高索引 577。

$\text{Eval}_g(d, x)$ 按描述符顺序扩展完整规范导线向量：

$$\nu[2 + m + i] = g(\nu[s_i^0], \nu[s_i^1]).$$

$\text{Enc}_U(d, x)$ 把规范描述符、输入和控制写入映射；WMC1 只是宿主序列化，不是 V 接受的 BPF 程序。固定解释器 P_U 读取配置并诱导 $\text{Sched}_U(d, x)$ 。状态屏蔽接口只在状态为 OK 时返回规范导线观察；这不声称物理陈旧单元被擦除。

E4-D (有界数据参数化解释)：要求同一个 P_U 与 E1–E3 门基共享制品，并满足：固定映射定义与记录环境中的接受独立于 d, x ；有效配置初始化常量和输入，恰按 $0, \dots, n - 1$ 执行并成功终止；一个外部临界区覆盖全部共享映射的设置、调用和回读；每次迭代验证规范形式、复制两个较早源、重置并调用门，只写规范目的与审计单元并保持描述符和既有导线；畸形配置在至多 512 次内以非 OK 终止并返回语义结果 $(s, \perp)$ 。

3.3 实现定理

定理 1 (显式义务下的有界组合)。若同一固定 P_U 在声明的确定、串行化环境下履行 E4-D，且其门基以函数 g 履行 E1–E3，则对所有 $d \in D_{64,512}$ 和 $x \in \{0, 1\}^{m(d)}$ ，

$$\text{Run}_{U,d}(\text{Enc}_U(d, x)) = (\text{OK}, \text{Eval}_g(d, x))$$

并恰执行 $n(d)$ 次迭代。若识别边界另有 Safe 的安全健全性前提，这些轨迹才可额外推出属于 Safe。

证明由规范前缀归纳给出：E4-D 初始化前缀与精确调度，描述符界保证源均已建立，E3 回到 q_0 ，E2 给出 g ，E4-D 只写新目的并保持前缀；E1 只负责把该功能位绑定到同后缀因果区分。定理分解证明义务，不声称测试枚举了描述符域，也不证明“每个电路生成一个新 BPF 对象”的 E4-A。

4. eBPF 校准案例

4.1 执行与载体边界

固定程序 $P_U = \text{wm_circuit}$ 的节为 $\text{SEC}(\text{"syscall"})$ 。记录环境中，内核验证器接受该对象，用户态经 BPF_PROG_RUN 接口离线执行；保留的宿主调用使用 libbpf 的 $\text{bpf_prog_test_run_opts}()$ 包装。Linux 文档说明该执行接口以及 syscall 节/程序类型映射 [10], [11]。该解释器运行没有实时钩子。验证单元是固定 P_U ，不是 WMC1 或描述符；接受后， P_U 读取映射并生成辅助函数操作与导线观察。

门映射 G0 是 BPF_MAP_TYPE_HASH ， $\text{max_entries}=2$ ，非 LRU，保持默认预分配 [12]。规约要求重置成功，并由外部临界区覆盖设置、调用与回读；程序本身没有并发锁。

4.2 饱和秩 NAND 与条件性 C

使用互异的哨兵键 S 与输入键 A, B 。重置删除三键后插入 S 。零更新 S ，第一个一插入 A ，第二个一插入 B 。在有效参数、默认预分配、成功重置、无干扰和记录 Linux 6.17 环境前提下，已有键更新成功且不减占用，新键在容量以下成功并增占用，容量已满时失败；只观察第二次更新是否成功，不依赖具体错误码 [12]。四种输出为 1, 1, 1, 0，即 NAND。NAND 具功能完备性，因为 $\neg x = \text{NAND}(x, x)$ 且 $x \wedge y = \neg \text{NAND}(x, y)$ 。

定义 1 的具体见证取内部前沿为第一次输入条件操作之后、第二次更新之前。令 $R_{G0}(\sigma)$ 为专用映射 G0 的**完整辅助函数相关动态状态**，包括键占用、桶、预分配元素/空闲链元数据以及更新辅助函数读取的任何其他映射局部状态，并取 $\rho_{\text{obs}}(\sigma) = R_{G0}(\sigma)$ 。占用键集合 K_{G0} 只是 R_{G0} 的派生投影，不能单独充当完整选定状态。

输入 $(0, 1)$ 与 $(1, 1)$ 在该前沿的派生占用分别为 $\{S\}$ 与 $\{S, A\}$ ，所以完整状态不同。公共后缀 w 是插入新键 B ，随后统一捕获并存储成功位； $\text{Obs}(\tau) = [\text{ret}(\tau) = 0]$ 。 ctx_w 包含程序点、键和值、G0 身份与静态属性、模式、相关控制/导线值，以及所有位于 R_{G0} 之外但被后缀读取的分量； Env 固定内核、对象、映射实例、调度及无干扰。在声明的映射更新契约下，两次观察为 1 与 0。较早输入不被后缀读取，其持续且与后缀相关的效应包含于完整 R_{G0} 中。因此这是**依赖具体服务契约的条件性 C** 见证，不是内核内部状态已被轨迹完整暴露的主张。

门规约的 $s_{D_G}(\sigma) = (\text{phase}(\sigma), K(\sigma))$ 只在由有效参数、专用预分配映射契约、成功重置、串行化与无干扰界定的 X_{D_G} 上使用。其操作充分性是显式服务契约前提，不是对所有内核字段的机器检查模型。E1 由上述内部前沿见证支持；命题 2 与重置前提支持 E2–E3。该机制不增加 eBPF 的外延布尔表达能力，也不单独证明报告遗漏。

4.3 固定解释器、前缀不变式与证据边界

解释器还使用 TAPE、CIRCUIT、WIRES、VM_CONTROL、VM_TRACE。宿主解析并规范化 WMC1，写映射，只在 $\text{status}=\text{OK}$ 后投影请求输出；输出列表不由 BPF 读取。内核中有界循环处理至多 512 个描述符；每次迭代先复制描述符和源值，再调用辅助函数、重置 G0、执行门并按键更新规范目的，避免跨辅助函数保留映射值指针。

源码级前缀不变式在下列前提下成立：命题 2 更新律、成功重置及必需调用、E4-D 串行化、有效编码，以及捕获目标保留人工检查的源码控制/数据依赖。 bpf_loop 请求 $n(d)$ 次迭代；回调返回 0 继续、1 停止，辅助函数返回执行次数 [13]；解释器只有在该返回值与自身完成计数均为 $n(d)$ 时接受。归纳得到每个规范前缀等于 $\text{Eval}_{\text{NAND}}$ 。这是源码级论证加人工检查转储，不是机器检查的 eBPF 语义证明。

因此：A 已记录；C 仅在声明的映射服务/无干扰契约下为条件性；P 还依赖源码到对象、帧保持和串行化前提。对这个 wm_circuit 载体，没有解释器前沿的 Linux 报告单元提取器或部署策略，故其 R 与 W 均未建立。

5. 评估

5.1 已记录环境与主要运行

主要证据位于 `results/interpreter/interpreter-final-20260711-02/`，环境为 Ubuntu 24.04、Linux 6.17.0-35-generic、aarch64。证据包保留四个 BPF 变体、已加载程序元数据、验证器日志、描述符、JSONL、作者运行的语义审计、当时选择的源码/手稿快照及自行签发的 SHA-256 清单。最终手稿晚于该运行，不受该清单覆盖；证据由作者生成并由作者另行审计，不是第三方复现。

证据行总数为

$$38,533 = 26,488 \text{ 条逐门记录} + 12,037 \text{ 条成功运行记录} + 8 \text{ 条负对照记录},$$

它们是异构证据行，不是“38,533 个测试”。

5.2 覆盖范围

覆盖包括: 9 个具名电路 (39 次成功、166 门记录); 100 个固定种子随机 DAG (1,876 次、23,776 门记录); 512 门深度与 64 输入/512 门/578 线联合边界 (4 次、2,048 门记录); 零门与 10,000 次串行调用 (10,001 次); 三种机制/基线对照 (117 次、498 门记录); 8 个畸形案例。联合 64 输入边界只测试全零和全一, 不是 2^{64} 穷举; 整个语料库是回归证据, 不是 $D_{64,512}$ 的枚举证明。

5.3 独立语义审计与完整性检查

独立语义审计重建具名和边界描述符、再生成随机语料、独立计算完整导线向量、检查目的与畸形案例并匹配运行标签; 审计报告成功。清单验证也成功, 但清单不是签名、时间戳、认证或独立复现。

5.4 机制归因

容量 64 与强制哨兵对照均使具名语料 166 个门输出全为 1; 算术基线产生预期结果。它们支持“容量饱和和第二个新键共同导致零输出”的机制归因, 不证明等价验证器报告。

5.5 校准结果

所引用的验证器概览把目标表述为程序安全、路径、参数和内存访问 [14]; 该概览本身未给出映射介导计算的完整功能证书。保留的 `wm_circuit` 证据在解释器前沿没有报告提取器, 因此不推断验证器单元跟踪该程序辅助函数返回值的精度。该载体用于校准 LangSec 中“已识别安全性”与“被解释功能”之间的边界: 其 R 仍未建立; 第 5.7 节独立评估 stock-Linux V1/V2 的 operational-prune 证据边界。

5.6 固定辅助可执行报告实例

为在不重命名 Linux 验证器制品的前提下执行报告相对判据, 将辅助 R 实例嵌入第 7.1 节的完整模型载体并固定

$$M_{linux_r_aux_v1} = (V_{linux_r}, I_{hash}, \text{Report}_{aux}, K_{obs}, P_{aux}, \ell, D, F, \emptyset, \emptyset, \emptyset, \emptyset).$$

P_{aux} 由自定义报告生成识别器 V_{linux_r} 接受; 它不是经 stock Linux 验证器接受的 BPF 对象。 I_{hash} 是有限、确定、串行且无干扰的受限语义, 只覆盖固定程序使用的非驱逐 HASH 映射更新情形。这里 Report_{aux} 只指 `report.json["report_cells"]`; `derivation.json` 仅记录工作列表/计算溯源, 不属于报告标签接口。最后四项分别是有类型的空行为主体集、空效果集、空驱动关系和空许可效果集; 它们只把 R 实例嵌入共同载体, 不提出 W 论断。

可执行证书。 $R(M_{linux_r_aux_v1})$ 成立 (制品状态: established)。

证书论证。 识别器接受 P_{aux° 在单一串行无干扰环境中穷尽 $a \in \{0, 1\}$ 、 $b = 1$, 得到 $F = \text{Reach}_{I_{hash}}(P_{aux}, \ell) = \{\text{frontier:S}, \text{frontier:AS}\} \subseteq X_D$; X_D 还包含相应的两个终止状态。令 $a_{obs} \in A_D$ 表示名为 `update-suffix-and-observe` 的规约动作, 以 c_{obs} 表示精确编码的运行时操作符号 `bpf_map_update_elem(G0, suffix_key, one, BPF_ANY); observe(ret==0)`, 并令 $\iota_{P_{aux}, \ell}(a_{obs}) = c_{obs}$; 这个单元元素编码是单射。 $\iota_{P_{aux}, \ell}(a_{obs})$ 的具体执行在两个前沿状态上有定义, 在终止状态上无定义, 并在定义性、输出和后继上与 D 一致, 故检查器在整个 X_D 上建立操作充分性。观测契约把 ρ_{obs} 固定为已占用键集合, 把 Obs 固定为有序成功位词, 把 Slice 固定为程序阶段/服务上下文对, 把 Env 固定为上述单一环境; 特别地, $\rho_{obs}(\text{frontier:S}) = \{S\} \neq \{S, A\} = \rho_{obs}(\text{frontier:AS})$ 。对于 F^2 中每个有序状态对和

$\mathcal{W}_D(F) = \{\varepsilon, a_{\text{obs}}\}$ 中每个词, 检查器均建立其余运行时间、共同上下文、观察者兼容性与 K_{obs} 健全性义务。这些义务履行了除唯一单元覆盖之外的全部可接纳条件。

Report_{aux} 发出一个唯一单元 $a^\#$, 共同覆盖两个前沿状态, 从而完成 $\text{Adm}(P_{aux}, \ell, D, F; K_{\text{obs}})$ 。精确有限未来观测商把二者分到不同类, 故 $|\beta_D(F_{a^\#})| = 2$, 且 a_{obs} 分别输出 1 与 0。同一后缀因此先见证实定义 1, 而带标签元组 $(P_{aux}, \ell, D, F, a^\#, a_{\text{obs}})$ 满足定义 2, 所以 $R(M_{\text{linux_r_aux_v1}})$ 成立。证据另含 21 项域/动作的**返回类别包含检查**和 2 项报告单元的后继**包含检查**, 均为零违反; 前 21 项不是 21 个完整状态检查。精确占用跟踪、容量 64、强制哨兵、忽略返回四个负对照均消除 R。□

在归档证据包上, 另行实现且由作者运行的检查器不导入模型实现, 重构可达性、唯一单元覆盖、商和因子分解判定, 并给出辅助 R 已建立的判定。另一个回归测试仅把同一确定性模型证据包构建两次并逐字节比较五个形式化 JSON 文件; 它是确定性测试, 不是实验性证据。该证书属于可执行有限模型证据, 不是机器检查证明。保留的内核校准对 (0, 1) 与 (1, 1) 两个赋值共有 4 行 oracle; 它只校准这两个受限服务结果, 不证明 I_{hash} 与 Linux 之间的 refinement 或 bisimulation, 也不提取 stock 验证器单元。因此辅助证书与下节 stock-Linux V1 证据记录彼此独立。其证据路径为 `results/linux_r/linux-r-v1/`。

5.7 Stock Linux 迹证据: V1 更正与 V2 proof-bound 结果

本节把已经完成的本地 stock-Linux V1 实验作为证据记录复核, 而不是轨迹局部 R 证书。内核捕获是一手证据: 它记录 frozen tuple 上一条 exact-level-0 `states_equal` 成功后由 `is_state_visited` 产生的 operational-prune 边, 以及同一后缀下两个已捕获运行的不同程序级观察。这是 MAY difference, 并未建立真实 Linux 行为的稳定 must outcome。

记 M_{Linux} 为该冻结 V1 元组上的实际 stock-Linux/对象/内核目标。它是 evidence-bounded query 的目标, 但 V1 记录没有为它提供完整转移关系、Linux 功能报告契约或 outcome-eligibility 证明。它不得与下文作者事后构造的有限适配器模型 $M_K^{\text{legacy}} = M_{\text{adapter}}$ 混同。

evidence-model v1 要求 outcome eligibility 后, V1 的 eligibility 为 NOT_ESTABLISHED。因此, 精确的 operational-prune query 结论为 UNKNOWN; 更广的运行或元组也为 UNKNOWN; Linux 功能报告为 OUT_OF_SCOPE。集成检查器与冻结包检查器只能验证声明的字节、哈希和捕获关系, 不能把作者构造的报告转化为 Linux 文档规定的功能报告契约, 也不能把单次捕获提升为真实 Linux 的 R 结论。

历史适配器记录 (不可作为当前结论)。V1 曾使用一个事后声明、受哈希绑定的两历史载体。该适配器把两个样本嵌入确定性的二状态、单动作模型 $M_K^{\text{legacy}} = M_{\text{adapter}}$, 并在该**构造模型**上得到因子分解失败。下列对象、转移和检查说明该旧适配器如何工作; 它们既不是 Linux 转移语义, 也不建立 $R(M_{\text{Linux}})$ 、定义 2 的 Linux 实例或“轨迹局部 R 证书”。

旧适配器固定的构造载体为:

$$M_K^{\text{legacy}} = M_{\text{adapter}} = (V_K, I_K, \text{Report}_K^{\text{prune}}, K_{\text{obs}}^K, P_R, \ell_K, D_K, F_K, \emptyset, \emptyset, \emptyset).$$

P_R 是固定且经验证器接受的 XDP 对象 `rac_single`, 不是解释器 $P_U = \text{wm_circuit}$ 。 V_K 是记录中的 stock Linux 6.17.0-35-generic 验证器, 精确比较级别为 0; $\ell_K = \text{pre}(41)$ 是共同 `shared_suffix` 调用前的调用者侧翻译后前沿。对这个事后命题, I_K 是在捕获完成后, 由论文把证据包中的两次已记录执行限制并加阶段标签而构造的有限串行关系。它既不是证据包适配器的自环转移系统, 也不是所有 Linux 执行的集合。其声明的输入/环境域只含两条已捕获选择器历史, 所以下面的可达性等式是受限分析载体的定义, 而不是关于 Linux 全部可达状态的经验完备性主张:

$$\text{Reach}_{I_K}(P_R, \ell_K) = F_K = \{\sigma_0, \sigma_1\}.$$

令 $X_{D_K} = \{\sigma_0, \sigma_1, \sigma_0^+, \sigma_1^+\}$; 这些符号表示针对两个前缀案例及其后缀后记录构造的阶段化具体化见证, 并非

完整运行时寄存器、栈和辅助函数内部映射状态的直接快照。两个 `runtime.json` 最终状态记录只绑定派生的 G0 键集合投影 $\{S, B\}$ 、 $\{S, A\}$ 及程序级结果；验证器捕获另行绑定选定的 State V2 记录。令 a_B 为抽象插入 B 动作，并定义宏符号 $c_B \in \Sigma_{\text{op}}(P_R)$ ，其声明的具体解释执行完全相同的剩余翻译后缀；置 $\iota_{P_R, \ell_K}(a_B) = c_B$ 。一步规约为

$$\begin{aligned} S_{D_K} &= \{p_0, p_1, q_0^{\text{term}}, q_1^{\text{term}}\}, \\ A_{D_K} &= \{a_B\}, \quad O_{D_K} = \{0, 1\}, \\ O_{\text{obs}}^K &= \{\varepsilon, 0, 1\}, \\ s_{D_K}(\sigma_i) &= p_i, \quad s_{D_K}(\sigma_i^+) = q_i^{\text{term}}, \quad \delta_{D_K}(p_i, a_B) = q_i^{\text{term}}, \\ \lambda_{D_K}(p_0, a_B) &= 1, \quad \lambda_{D_K}(p_1, a_B) = 0. \end{aligned}$$

在具体层面，声明的关系对该动作恰好包含

$$\sigma_i \xrightarrow{c_B/b_i} I_K \sigma_i^+, \quad (b_0, b_1) = (1, 0),$$

且从 σ_0^+, σ_1^+ 均无 c_B 转移。这些转移属于论文定义的包装；证据包中的 `runtime.json` 记录只绑定键集合投影与程序级结果，不绑定完整具体端点。从 $q_0^{\text{term}}, q_1^{\text{term}}$ 没有出边。两个观测者均返回 O_{obs}^K 中的位词： $\text{Obs}_{D_K}(\varepsilon) = \varepsilon$ 且 $\text{Obs}_{D_K}(b_i) = b_i$ ；在相应的声明具体侧， Obs 把空迹映为 ε ，把每条已记录 c_B 迹映为 b_i 。因此观测兼容性覆盖 $\mathcal{W}_{D_K}(F_K) = \{\varepsilon, a_B\}$ 中两个词。阶段标签和所列转移使操作充分及运行词包含在这个受限关系中按构造成立；它们并未独立验证更大的 Linux 转移语义。

stock 专用契约 K_{obs}^K 置 $\rho_{\text{obs}}^K(\sigma) = R_{G_0}(\sigma)$ ，即第 4.2 节定义的完整辅助函数相关 G0 动态状态，并观察程序级成功位词。记录的键集合只是派生投影 $K_{G_0}: K_{G_0}(\sigma_0) = \{S\}$ 且 $K_{G_0}(\sigma_1) = \{S, A\}$ ，所以即使记录没有暴露每个内部字段，也有 $R_{G_0}(\sigma_0) \neq R_{G_0}(\sigma_1)$ 。上下文固定调用方阶段与后缀、内核/对象/程序及映射身份、键值常量、映射类型/容量/标志、串行化、单制品条件以及 R_{G_0} 之外所有被后缀读取的分量。检查器比较声明的七个规范化运行时上下文字段；另由论文审查翻译后 PC 41--44 的调用方调用/返回路径及 PC 107--122 的 `shared_suffix` 被调用函数读集。被调用函数会覆盖键/值工作寄存器和栈槽；在辅助函数之外，它除此之外只读取固定常量与固定的 G0 映射身份，而调用方仅调用该函数并返回其结果。映射局部桶、元素/空闲链元数据及辅助函数读取的其他 G0 状态属于 R_{G_0} ；哈希绑定的单元元素 `Env` 固定内核、对象、精确映射实例、服务/分配选择、调度与无干扰。因此两条记录具有相同的非选定后缀读取上下文。由于选定状态的投影已不同，契约在 F_K 上“ R_{G_0} 相等”的前提只会把一个状态与自身配对；声明的一步服务关系具有确定性，故契约健全。记录的成功位等于 Obs_{D_K} ，故观察者兼容。这样，运行词、共同上下文、操作充分、观察者兼容和健全性义务只在这个显式有限载体上得到履行，而非在一般 Linux 执行上得到履行。该读集论证以及 $I_K, D_K, K_{\text{obs}}^K$ 的定型是论文审查的义务，不是对检查器机器验证范围的扩大表述。

$\text{Report}_K^{\text{prune}}$ 是作者为分析该捕获事件而显式选定的**操作剪枝报告**：`states_equal` 成功检查实际导致 `is_state_visited` 剪除 current state 时，它把可接纳前缀见证分配给 retained verifier-state representative。有向剪枝边就是成员关系；本文不把 `states_equal` 重释为完整具体状态上的对称数学等价。所引用的验证器概览把验证器描述为安全机制；该概览本身未把此接口规定为覆盖运行时映射占用的完备功能报告。因此这个报告是从具体剪枝事件提取的分析投影，不是已被实验推翻的 Linux 指定证书。

冻结清单绑定内核版本、BTF 与配置哈希、对象 SHA-256、已加载程序 id/tag/pin 以及翻译后字节码 SHA-256。选定 `fexit` 事件在指令 41 同时记录 `states_equal_success=true` 与 `is_state_visited_prune=true`。retained 与 current 的历史不同，分别经过 $a = 1$ 与 $a = 0$ 对应的翻译后分支调用，再到达同一前沿；二者状态哈希也不同（516c47f044cc3fc3 与 a66d912d58c91de4），成员关系来自观测到的有向剪枝边，而非哈希相等。经审查的路径相关检查器把两条历史绑定到该前沿和同一剩余字节码后缀；捕获完成记录显示丢失事件与解析错误均为 0。

规范化成员检查把构造见证 σ_0 与捕获的 current verifier state 关联，把 σ_1 与捕获的 retained verifier

state 关联；这些检查并未捕获完整运行时前沿状态。运行记录把见证的派生 G0 键集合投影分别绑定为 $\{S\}$ 与 $\{S, A\}$ 。记 $a_K^\# = \text{retained:516c47f044cc3fc3}$ 。在受限 exact-0 State V2 形状和声明的操作报告内，形状检查器与唯一单元检查器建立

$$\gamma_{\ell_K}^K(a_K^\#) \cap F_K = \{\sigma_0, \sigma_1\}.$$

而且在 F_K 上，两状态都不属于任何其他已提取报告单元。因此

$$\pi_R(\sigma_0) = \pi_R(\sigma_1) = a_K^\#.$$

在共同定义的词 a_B 下，记录的**程序级成功位**从两个已捕获案例出发分别为 1 与 0；这两个位由 `observe(ret==0)` 派生，不是映射更新辅助函数的原始返回值（成功为 0、失败为负值）。旧适配器据此把样本放入其自定义的二状态、单动作模型 $M_K^{\text{legacy}} = M_{\text{adapter}}$ ，并在那个模型上得到不同的未来观测类和因子分解失败。该结果只描述旧适配器的构造模型，不能证明 M_{Linux} 具有相同的状态空间、转移、报告单元、定义性或稳定结果。

因此，当前结论**不是** $R(M_{\text{Linux}})$ ，也不是定义 2 的 M_{Linux} 实例，更不是“事后、轨迹局部、相对于操作剪枝报告的 R 证书”。因子分解结果只属于历史构造 $M_K^{\text{legacy}} = M_{\text{adapter}}$ 。在 evidence-model v1 下，V1 的 `outcome_eligibility` 为 NOT_ESTABLISHED；精确 operational-prune query 为 UNKNOWN，更广的运行/元组为 UNKNOWN，Linux 功能报告为 OUT_OF_SCOPE。这也意味着不得把 $I_K, D_K, K_{\text{obs}}^K$ 的事后定型、 β 不等式或任何旧适配器因子分解检查，外推为真实 Linux 语义结论。

冻结包中的 STOCK_LINUX_R_ESTABLISHED_FOR_FROZEN_TUPLE 和 FROZEN_PROOF_BUNDLE_VERIFIED 是历史输出/字节完整性标记：前者记录旧适配器曾对冻结输入通过其哈希门，后者验证不可变包的字节与哈希。二者均不改变上述当前语义判定。冻结证据位于 `residuality-auditor/stock-linux-r-proof/`；若要升级结论，必须先满足 evidence-model v1 的 `outcome-eligibility` 要求，并为真实 Linux 提供独立、预先声明的转移和报告契约。

V2 是独立的前瞻实验，不是对 V1 元组的重释。令 M_{Linux}^{V2} 表示由 `residuality-auditor/linux/scripts/run_stock_r_v2.sh` 针对已接受 `rac_v2` witness 生成的受控 stock-Linux/对象/内核/映射实例。V2 契约预先声明 `outcome-free` 的 verifier-prune query、invocation-scoped fentry/fexit 捕获、array-map witness、固定源码/构建闭包和 proof checker。runner 先封存 runtime identity，再写出 `proof/must-outcome-proof.json` 与 `proof/history-case-binding.json`，最后审计整个 bundle。证明文档绑定 query digest、源码与构建闭包、对象 SHA-256、翻译后字节码 digest、BTF/内核标识、runtime-event identity、checker calculus 与 checker source closure；binding 将 retained/current 历史 digest 分别连接到 proof case 0/1，并固定同一前沿、报告单元、后缀、观察者与 exact-scope digest。

V2 的 `must-outcome proof` 补上 V1 的模式缺口，`history-case binding` 则补上该证明与实际选中剪枝历史之间的连接缺口。检查器并不把“运行时出现两个不同结果”本身当作证明；它在 `stock-r-v2-array-map-must-outcome-v1` 演算中重放两个 case derivation：低输入位决定 array-map slot 0 的更新，随后 lookup 读取 slot 0，程序返回该低位。因此派生 case map 为 $\{0 \mapsto 0, 1 \mapsto 1\}$ 。只有当这些派生结果与重复运行记录一致、观测到的 prune 是声明的 operational-prune 事件、`history-case join` 验证成功且所有 identity receipt 都匹配 sealed manifest 时，bundle 才被接受。证明有效但 binding 缺失时仍为 UNKNOWN/NOT_ESTABLISHED；证明、身份、binding 或运行时证据缺失/畸形时均 fail closed。

在这些 gate 下，一次新的 stock Ubuntu 6.17.0-35-generic 特权运行得到：`outcome_eligibility.status=ESTABLISHED`、`outcome_eligibility.method=MUST_OUTCOME_PROOF_WITH_HISTORY_CASE_BINDING`、`assessment.status=NONFACTORING`、`assessment.scope=EXACT_STOCK_R_V2_QUERY`，证书为 `NONFACTORING@1d5f86d80494575c23f539248614105559dd15380c580d0d2388c24941b6d255`。这是受控 V2 witness 上针对声明 operational-prune report 的报告因子分解结果。它仍不建立 Linux 指定功

能报告、一般 `states_equal` 语义、其他输入或后缀、其他内核或对象、验证器不健全、漏洞、P、W 或怪异机器。它也不回填 V1: 冻结 V1 的 exact query 仍为 UNKNOWN。

U4 参考层把源证据验证与通用授权分开。V1/V2 专用适配器验证原始 bundle 并编译出版本化 claim、带类型 evidence graph 与 proof DAG; 通用检查器不读取 bundle 的终局 verdict, 而检查节点 digest、图的顺序/依赖、证明引用与精确范围规则。在保留证据上, V1 得到 BLOCKED/INCONCLUSIVE, 其 strongest profile 只有 MAY_OUTCOME 与 REPORT_COLLISION; V2 得到量词 AT、权威 OPERATIONAL_OBSERVATION、等级 OUTCOME_FREE_PRECOMMITTED 的 CERTIFIED/NONFACTORING。敌对矩阵把请求层的 FORALL、指定报告、观察者、后缀和 TRANSPORTED 五种提升全部阻断; 证明整体自洽改写量词、报告权威、证据等级或报告关系, 以及 outcome-to-selector 依赖、payload 篡改和缺失 proof premise 七种攻击均被判为无效。这只是 exact-scope 论断纪律的可执行控制, 不是类型化 scope transport、family cover、编译器正确性或新的基础理论。

CRL 在 verifier-contract 层增加一个受保护的运输定理。抽象契约

$$V = (S, H, \pi, equal, visited, step, \eta, \rho)$$

中, `StockR_V(s,w)` 表示两个源历史在同一报告单元中碰撞, 并在同一后缀下具有 singleton 且观察者可区分的结果。若被接纳的上下文项 `C` 是 transparent——包括见证区域上的全指令对应、footprint/effect 不相交、碰撞/后缀/must-outcome/观察者/前沿/报告单元/history-map 保持、target-conformance bridge、选择与结果无关、且不把目标终局 verdict 当作前提——则得到 exact target scope 上的 `StockR_V(t,C(w))`。可执行证书是 DERIVED_CONTEXTUAL 链; 检查器重算 source claim、transform 与 target claim digest。底层 EBRC 规则仍为 CONTEXT_TRANSPORT, 唯一被授权的目标结论是 evidence grade 为 TRANSPORTED 的 AT(target)。

保留的 U6 VM matrix 在冻结的 BOUNDED_CONTEXT_SUITE_ONLY suite 上实例化该定理。公开运行 contextual-matrix-live-20260720-03 包含 12 个符合预期的 case: 6 个 transparent context 被认证, 6 个 hostile 或缺义务 context fail closed。replay capsule 选入两个 transparent 目标: `transparent.xor.depth1` 与 `transparent.add-mul.depth2`。两个目标翻译后字节码 digest 分别为 `c902feca11d3825fa1317fab7605312795d4531d83e26aa653fcc214632217de` 与 `e36423e7fcf712ee187636fd0dbe3a91fafdbd82b85f60b6ee4cbe85df49adad`, 均不同于 V2 源 digest 且彼此不同。目标对象 hash 分别为 `84c0ba6fc0d1a702dded1037f7e782d51d3713b54221e67db4474e0aaf6ac531` 与 `13902d95bac31c2fe861dccc24a58b5e3b4d43f7d4b90a0165bf85a1310c7a02`; 两个 runtime bridge 均在四次 trial 上为 VERIFIED, contextual checker 发出 NONFACTORING@23b72c129e12520df1b05580f4ab74582f49b4e4f442db3e05e207a7deffc1e2 与 NONFACTORING@1d19c14f69a186648acae58c57c68faf7b9719a51b2e870e53baf81efc663。每个 contextual hostile matrix 均为 `all_expected = true`, 包含 3 个 blocked unsupported claims 与 9 个 invalid graphs, 其中包括 stale/rewritten DERIVED_CONTEXTUAL 链。这些只是 target-bound transport instances, 不是 FORALL context theorem、编译器正确性证明、Linux 功能报告、验证器漏洞、P、W 或怪异机器结果。

5.8 有效性威胁

实验只使用一个内核构建版本和一种体系结构。论证依赖专用的预分配非 LRU 映射、成功重置、互异的键、所述更新律, 以及整个映射集合上的互斥。会发出门记录的数据集保留原始返回值; 10,000 次运行的压力测试文件记录运行/状态/输出证据, 但不包含逐门原始行。证明只使用成功与失败之分, 并不承诺可移植的错误码。

有限测试不能证明所有 $D_{64,512}$ 。定理 1 转而依赖所陈述的源代码级初始化、验证、有序迭代、门和帧义务; 语料库旨在寻找反例。翻译后转储和验证器日志被保留下来以供人工检查及清单哈希使用, 而未被自动化数据流检查器消费。状态掩蔽具有源代码顺序保护, 而负面测试套件测试的是拒绝/状态, 并非注入的掩蔽路径。不存在机器检查的 eBPF 语义证明, 且只有在可选的 Safe 结论中才假定生产验证器的安全健全性。

stock-Linux 证据是在隔离内核/对象元组上由作者生成并审查, 尚未得到第三方独立复现。对 V1 而言, 旧操

作报告、两历史载体、观测者和后缀都是事后最终确定，因此即使内部检查全部通过，结果仍受选择效应威胁。V1 证据包绑定 BTF、配置与源码映射元数据，但没有保存目标 `verifier.c` 函数正文；受限形状引理不是 Linux 一般 `states_equal` 语义的源码级定理。对 V2 而言，前瞻 runner、must proof 与 history-case binding 移除了“缺少 must outcome”和 witness join 这两个特定缺口，但证明演算仍是小型源码级微语义检查器，不是 Linux、libbpf、JIT 代码、helper 内部或验证器源码的机器检查语义。CRL 增加的是模型契约运输定理与两个生成式目标实例，仍信任源专用适配器和声明的 target-conformance bridge。通用检查器只验证已编译的有限 fragment；解释原始 bundle 字节仍信任源专用适配器。仅凭原始事件、状态哈希、旧适配器模型、没有 proof 的重复运行差异、没有 bridge 的字节码相似性或类似验证器日志文本，均不能满足真实 Linux 的定义 2 前提。

原始捕获阶段分析器有意保持 fail-closed：在字节码前沿、路径对应、报告契约和具体化关系尚未审查时，它输出 `LINUX_R_NOT_ESTABLISHED_FROM_RAW_CAPTURE`。旧集成检查器随后消费经审查的历史适配器输入并产生元组特定的历史输出；evidence-model v1 不接受该输出为正式 R 结论。仅凭原始事件、状态哈希、旧适配器模型、没有 proof 的重复运行差异或类似验证器日志文本，均不能满足真实 Linux 的定义 2 前提。

6. 相关工作

语言理论安全提供识别/解释框架 [1]–[4]；对象中心追踪重建状态依赖的底层操作语言 [15]。怪异机器研究非预期计算与可利用性 [16], [17]，不安全编译显式引入策略边界 [18]，携带证明代码工作识别证明模型之外的影子执行 [19]。有效 RPM 元数据也可产生计算，漏洞流类型可导出抽象怪异机器 [20], [21]。不同于只展示丰富行为的工作，本文 R 判据追问的是：在一个已接纳制品的前沿上，已声明的实际计算报告是否真的因子分解未来观测商。

已发表机制横跨不同载体。代码复用和数据导向攻击利用现存指令、信号恢复、非控制数据、对象分派或异常处理 [35]–[39]；后续工作还组合彼此不兼容的语言防御，或综合出可实现的 DOP 攻击语言 [52], [53]。DWARF 与 ELF 元数据驱动辅助解释器和 RTLD [40], [41]，携带证明代码则暴露证明模型之外的 proof-aliasing 执行 [19]。页故障构造在没有成功指令分派时使用 IA32 转换与故障机制，而且原文明确说它既不是漏洞也不是 exploit [42]。内存去重结合 Rowhammer 得到可利用机器 [43]；FORCEDENTRY 技术分析还重建了一个在野 JBIG2 解析器机器：超过 70,000 条逻辑 segment 命令组成小型虚拟体系结构 [51]；MS11-087 报告则描述恶意 TrueType 字体驱动内核渲染器产生攻击者选择的内存效果 [54]。推测与微体系结构工作把计算藏入瞬态或时序状态，并进一步编译高层程序 [44]–[46]。packet-in-packet 跨协议层解释，中断导向工作刻意研究弱于通用计算的机器，BGP 配置则在给定假设下实现逻辑电路 [47]–[49]。这些差异正是第 7 节 P/W 分类格的依据：计算表达力与策略效果不能互换。

抽象解释提供具体/抽象、健全性和完备性词汇 [5], [22]；本文只讨论一个前沿上唯一分配的实际计算单元。MOAT 为应对基于验证器的安全性局限，使用 MPK 隔离潜在恶意的 BPF 程序 [23]，mismorphism 比较不同解释 [24]。eBPF 范围分析验证与状态嵌入针对验证器逻辑的健全性或覆盖错误 [25], [26]。VEP 所谓“programmability”是减少验证工具链限制；Rex 通过语言级安全与语言外运行时替代独立静态验证器，处理拒绝侧的语言—验证器失配 [27], [28]。本文 P 不同：它要求一个经原验证器接受的固定制品解释有界族。DRACO 在验证器接受后检查功能规范；bpfverify 把 eBPF 字节码翻译为位与内存精确的 Horn 子句以进行功能验证 [29], [30]。

2026 年预印本进一步界定边界：Heimdall 处理编译和接受后仍存的高层缺陷；Yaksha-Prashna 提取第三方 eBPF 字节码行为；bpfix 定位被拒程序中的证明丢失；信任边界语义缝隙工作分析正确语法接受后仍不足的断言 [31]–[34]。它们都未联合区分本文论断图谱的全部义务。反过来，本文 `wm_circuit` 解释器载体也未建立 R 或 W；辅助元组只在其自定义载体上建立 R；Stock Linux V1 只保留 operational-prune/MAY 记录并得到 UNKNOWN；V2 只建立 exact-query operational-prune NONFACTORING。四项 2026 工作均按预印本引用，而非声称已经同行评审。

7. 局限、启示与展望

7.1 严格区分与分类谓词

令模型

$$M = (V, I, \text{Report}, K_{\text{obs}}, P_*, \ell_*, D_R, F_R, \mathcal{A}, \mathcal{D}_A, \mathcal{E}, \text{Induce}, \text{Drive}, \text{Pol}, \text{Int})$$

还带有具名的安全、报告抽象与粒度契约。 $\mathcal{A}$ 、 $\mathcal{D}_A$ 、 $\mathcal{E}$ 分别为行为主体、驱动描述符和效果集合；**Induce** 记录主体/描述符可诱导的带标签操作词；**Drive** $\subseteq \mathcal{A} \times \mathcal{D}_A \times \mathcal{E}$ 记录效果；**Pol** 是许可效果集合；**Int** 是具名的预期解释契约。**A**、**C**、**P**、**R**、**W** 分别按前文定义，其中 $W(M)$ 要求某个 **Drive** 效果不在 **Pol** 中；**P** 还要求同制品门基满足 E1–E3、 P_* 履行 E4–D，且 g 与常量 $\{0, 1\}$ 构成功能完备的布尔基；**R** 的带标签见证必须使用同一个可接纳元组。

$\text{Ctrl}_C(M)$ 要求某个主体选择的描述符可诱导一个 **C** 见证， $\text{Unint}(M)$ 表示该解释在 **Int** 之外。 Link_{CP} 把可控 **C** 见证绑定到建立 **P** 的执行族， Link_{CW} 把 **W** 效果绑定到诱导 **C** 的主体/描述符族， Link_{PR} 把 **R** 碰撞绑定到 **P** 执行族， Link_{PW} 进一步要求 **W** 效果来自同一编码 **P** 计算。定义

$$\begin{aligned} \text{WM}_{\text{emergent}}(M) &= C(M) \wedge \text{Ctrl}_C(M) \wedge \text{Unint}(M), \\ \text{WM}_{\text{prog}}(M) &= \text{WM}_{\text{emergent}}(M) \wedge P(M) \wedge \text{Link}_{CP}(M), \\ \text{WM}_{\text{policy}}(M) &= \text{WM}_{\text{emergent}}(M) \wedge W(M) \wedge \text{Link}_{CW}(M). \end{aligned}$$

$\text{Doc}(M)$ 表示同一链接行为使用已文档化语义且维持声明的安全契约； $\text{Conf}(M)$ 表示报告实现符合指定抽象； $\text{Gran}(M)$ 表示 **R** 碰撞由声明的报告粒度强制。更窄分类为

$$\begin{aligned} \text{WM}_{\text{shape}}(M) &= \text{WM}_{\text{prog}}(M) \wedge \text{WM}_{\text{policy}}(M) \wedge R(M) \\ &\quad \wedge \text{Link}_{PR}(M) \wedge \text{Link}_{PW}(M) \\ &\quad \wedge \text{Doc}(M) \wedge \text{Conf}(M) \wedge \text{Gran}(M). \end{aligned}$$

定义直接给出 $R \Rightarrow C \Rightarrow A$ 与 $P \Rightarrow C \Rightarrow A$ 。有限反模型表明

$$A \not\Rightarrow C, C \not\Rightarrow P, C \not\Rightarrow R, P \not\Rightarrow R, R \not\Rightarrow P, P \not\Rightarrow W, R \not\Rightarrow W.$$

因此裸 **C**、抽象解释不完备或接受不完备都不推出 **P** 或怪异机器分类；策略级怪异机器也可能在报告精确时成立，故一般不要求 **R**。

命题 4 (严格怪异机器分类格)。定义直接给出

$$\text{WM}_{\text{shape}} \Rightarrow \text{WM}_{\text{prog}} \wedge \text{WM}_{\text{policy}}, \quad \text{WM}_{\text{prog}} \Rightarrow \text{WM}_{\text{emergent}}, \quad \text{WM}_{\text{policy}} \Rightarrow \text{WM}_{\text{emergent}}.$$

WM_{prog} 与 $\text{WM}_{\text{policy}}$ 互不蕴含，而且 $\text{WM}_{\text{emergent}}$ 、 WM_{prog} 、 $\text{WM}_{\text{policy}}$ 均不蕴含 **R**。证明使用有限反模型：把可控、非预期 E1–E4–D NAND 解释器的全部效果纳入许可集合，得到 **prog** 而非 **policy**；一个可控、非预期、一次性状态介导的禁用效果得到 **policy** 而非 **prog**；删去禁用效果得到只在基类的见证；再为各构造配置精确未来等价报告单元即可令 **R** 为假，而不改变其余谓词。正向蕴含由定义成立。

7.2 已发表怪异机器案例族的回溯覆盖

本文只把作者维护的 Weird Machines HQ 索引用于发现线索 [50], 随后核对 ACM、IEEE、NDSS、USENIX、Project Zero 与所引会议的原论文或作者第一手技术报告; 检索冻结日期为 2026-07-20。纳入条件是: 明确构造非预期计算, 或直接论证怪异机器机制, 且载体与驱动边界可辨认。正常且符合意图的虚拟机、唯一事实只是图灵完备的演示、无原始证据博客、仅比喻性使用该术语的工作, 以及没有引入新载体的防御论文均被排除。相同载体论证按族合并: Framing Signals 归入 SROP, Phantom Boundaries 与 802.3 injection 归入 packet-in-packet, DOPPLER 归入 DOP, TrueType 归入解析器/渲染器机器; 携带证明代码、FORCEDENTRY/JBIG2 与跨语言攻击则在表中明确列出。预期内的 RarVM 或 x86-MOV 计算、游戏清单与 accidental-Turing-completeness 清单, 在没有额外边界论证时不能通过纳入条件或 Unint 门。因此下表是具名已发表案例族的结构化回溯, 而不是声称枚举了历史上的每一个 exploit。

“候选”仅表示原论文报告了表中成分, 不表示本文已从原制品重建定义 1、E1–E4-D、W 或定义 2。尤其是, 下列论文均没有提供本文 R 所需的已计算报告单元提取器与唯一纤维因子分解证据。

已发表案例族	边界制品与被诱导引擎	原文报告能力	回溯分类与仍缺义务
x86 ROP、SROP [35], [36]	被破坏的栈词或信号帧驱动 现存 gadget 或内核信号恢复	通用/可移植计算及具体 exploit/backdoor	在其 exploit 上下文中是 $WM_{prog} \cap WM_{policy}$ 候选; 未重建精确 C/linkage 与 R
DOP、COOP、CHOP、CLA、DOPPLER [37]–[39], [52], [53]	非控制数据、伪造对象、异常状态或跨语言转移驱动数据流、分派或展开	表达性 DOP/COOP; CHOP 效果; 跨两个单独 阻断攻击边界的 CLA 控制 劫持; 可实现 DOP 语言综合	DOP/COOP 是交汇候选; CHOP/CLA 无需新 P 即支持 policy; DOPPLER 加强族级可 实现性; 无 R
DWARF、ELF、RPM、PCC [40], [41], [20], [19]	元数据驱动展开字节码、 RTLD 或包管理器自动机; 证明抽象接纳未计入执行	解释器与 Trojan/exploit 构造; PCC 的 proof-aliasing/未指定计算	DWARF/ELF 是 prog 候选; PCC 只有实例化主体与禁用效果 后才是 emergent/policy; 无 R
IA32 page-fault 机器 [42]	页表、IDT/TSS 与触发故障的 内存布局驱动转换、页故障与双故障	无成功 CPU 指令分派的图 灵完备计算; 原文明示既非 漏洞也非 exploit	典型 prog 候选, 不自动得到 policy; 无 R
内存去重加 Rowhammer; True- Type/JBIG2 [43], [54], [51]	选定内容驱动去重/物理故障, 或字体/解析器命令驱动内核/ 无界内存上的渲染与位图逻辑	任意读写/浏览器攻击; 渲染器 破坏; 通用门、虚拟体系结构 与在野 sandbox exploit	JBIG2 是两分支交汇候选; 内存 去重/TrueType 至少是无需新 P 证明的 policy 候选; 未重建精 确 linkage 与 R
ExSpectre、 μ WM、Flexo [44]–[46]	推测路径或微体系结构/时序 状态驱动隐藏门和编译电路	隐藏任意计算、电路构造、 高层编译器与恶意打包演示	prog 候选; 只有另行固定恶意/ 规避策略后才是 policy; 无 R
packet-in- packet [47]	合法外层流量中的内层物理 帧驱动第二种成帧解释	注入原本被禁止的无线帧	无需 P 的 policy 候选; 未重建精 确 C 与 R
interrupt- oriented bugdoor [48]	中断时序/嵌套与固件副作 用驱动极简嵌入式机器	以弱原语获得平台控制; 原 文明确对比最大表达力	$WM_{policy} \not\Rightarrow WM_{prog}$ 的典型动 机; 无 R

已发表案例族	边界制品与被诱导引擎	原文报告能力	回溯分类与仍缺义务
BGP 逻辑电路 [49]	路由配置与消息传播驱动门、 时钟和触发器	在给定假设下实现任意电路 并达到图灵等价能力	仅有 P 类表达力；在 Unint 以及 policy 所需 W/linkage 建立前 不推出任何怪异机器类别

页故障案例因此不是理论例外：控制结构诱导计算，使它落入可编程分支；而原文的“不是漏洞/不是 exploit”恰好阻止把它自动提升为策略分支。相反，中断与 packet-in-packet 说明，若强制所有策略相关怪异机器都先满足 P，就会排除原文刻意强调的低表达力但有实际效果的机制。

7.3 防御启示

审计时应分别声明识别属性和报告，枚举行为主体可驱动的操作与环境假设，再测试报告因子分解。契约违反需要修实现；已文档化碰撞只在报告确实意图认证该关系时，才要求细化、限制或隔离。

7.4 eBPF 怪异机器状态与未来形状定理

wm_circuit 载体在源码/对象和串行化前提下支持 P，但没有建立 Ctrl_C/Unint、W 或 R，所以 P 本身不产生分类。Stock Linux V1 的 M_{Linux} 记录只保存 operational-prune 边和 MAY 差异；旧 $M_K^{legacy} = M_{adapter}$ 构造模型的因子分解失败不得转写为 M_{Linux} 的 R 谓词、定义 2 实例或轨迹局部证书。其真实 Linux exact query 为 UNKNOWN。V2 的 M_{Linux}^{V2} 只对 EXACT_STOCK_R_V2_QUERY 建立 proof-bound NONFACTORING；其报告载体、对象和 witness 仍不是解释器载体。辅助模型 $M_{linux_r_aux_v1}$ 只针对固定自定义报告和受限服务语义建立 R。由于辅助和 V2 的 R 载体都不同于解释器 P 证书，Link_{PR} 禁止合并。故没有已评估 eBPF 模型建立 WM_{emergent}、WM_{prog}、WM_{policy} 或 WM_{shape}。未来形状定理必须在同一载体上推出接受的宏闭包、主体/链接义务与碰撞的报告嵌入；上下文敏感性可能破坏其中任一条件。

8. 结论

在分层的 LangSec 边界上，制品识别、诱导解释、报告因子分解、有界可编程性和策略效果是不同判断： $R \Rightarrow C \Rightarrow A$ 且 $P \Rightarrow C \Rightarrow A$ ；命题 3 用有限反模型否定其余列出的蕴含。命题 4 把主体可控、非预期的 C 放在严格分类格底座，并以可编程性和策略活跃性作独立细化。案例回溯因此可同时解释 x86 复用、元数据与证明模型缺口、page-fault、解析器与跨语言机器、极简 exploit、跨层解释、微体系结构机器与 BGP，而不混同图灵完备性、利用性或报告不可因子分解。

wm_circuit 载体记录 A，条件性见证 C，并在额外前提下支持固定 64 输入/512 门 NAND 解释器的 P，但不建立 R、W 或分类所需意图/控制谓词。辅助元组独立针对自定义报告建立 R。Stock Linux V1 捕获提供真实 exact-level-0 operational-prune 边以及同一后缀下的两个已捕获程序级结果；旧适配器仅在构造的二状态、单动作模型上显示因子分解失败，V1 exact query 仍为 UNKNOWN。Stock-R V2 针对独立受控 witness 补出 must-outcome proof 与 history-case binding，只对 EXACT_STOCK_R_V2_QUERY 建立 NONFACTORING；通用检查器保持该精确边界并拒绝测试过的越权提升。CRL 把该 exact 源证书运输到两个生成式目标，但只得到 exact AT(target) 证书，不得到 family theorem。辅助、V2 与 CRL 载体都不提供 P 或 W，也不与解释器链接，故 eBPF 证据不建立任何怪异机器类别、验证器不健全、漏洞或普遍必要性定理。

伦理与数据可用性

实验在隔离本地 VM 中运行, 不涉及第三方目标或生产数据路径。解释器运行没有实时内核钩子; stock-Linux V1 捕获只在隔离程序加载期间短暂把 fexit 观察器附着到验证器内部函数, 观察验证器决定而不改变已接受程序的执行。任何实验都没有尝试破坏、验证器绕过或提权。

仓库为 <https://github.com/Emtanling/eBPF-machine>; 不可变 eBPF 解释器证据位于提交 4309069a 下的 `results/interpreter/interpreter-final-20260711-02/`, 辅助报告实例证据位于提交 f665b1a 下的 `results/linux_r/linux-r-v1/`。Stock Linux V1 捕获发布在带标签的 V1.0 快照中, 路径为 `residuality-auditor/stock-linux-r-proof/`。其 `MANIFEST.json`、`CHECKSUMS.sha256`、嵌入输入哈希、原始捕获、规范化历史适配器输出和检查器源码共同纳入该公开快照。在 `residuality-auditor/` 下运行 `PYTHONPATH=.python3-mtools.proof.check_frozen_bundlestock-linux-r-proof` 会输出 `FROZEN_PROOF_BUNDLE_VERIFIED`; 它只验证冻结包的字节/哈希完整性, 不验证当前的 Linux R 结论。Stock-R V2、EBRC U4 与 CRL U5/U6 的源码、runner、checker 与测试位于 `residuality-auditor/linux/`、`residuality-auditor/src/residuality_auditor/` 和 `residuality-auditor/tests/`; `maketest-ebrc` 运行通用控制, `maketest-ebrc-context` 运行上下文控制, `maketest-stock-r-tools` 在记录的 VM 环境中通过 172 项测试。公开 replay capsule 位于 `residuality-auditor/artifact/evidence/replay-capsule.tar.xz`, SHA-256 为 3df6b96e3dded26e9f876db8f607278bc0a65a6df31b297cb6bd3043f44151f7, 大小 2,208,232 字节。`makereproduce-paper` 验证 capsule 并将重放得到的 V1/V2/CRL 结果与 `residuality-auditor/artifact/expected-results.json` 比较; `makecontextual-matrix-live` 在新的输出目录中重跑特权 `BOUNDED_CONTEXT_SUITE_ONLY` VM matrix。发布前归档 `residuality-auditor-v0.3.0-full.tar.gz` 的 SHA-256 为 5fd0a2812c8c8db2fe5508440934817c1cf9293ba0c5df31317e8b38d94a90ec; V1.0 直接发布冻结载荷, 不重复存储该归档。任何冻结字节变化都需要新版本、更新校验和并重新验证。

致谢与 AI 使用声明

OpenAI Codex 协助全文起草与语言修订, 并在作者指导下对摘要、引言、第 5.6--5.8 节和结论进行实质修订, 同时协助制品代码/测试修订及检查。作者独立审查主张、证明、引用、更改与结果并承担全部责任。作者声明不存在利益冲突。

参考文献

为避免中文阅读版形成第二套书目真源, 本文件不重复排印完整书目; 文内采用英文公开技术报告的 54 项显示序号, 完整、canonical bibliography 的书目信息与顺序以英文 `PAPER_REPORT.tex` 的 `thebibliography` 为准。[6]–[9] 为 Nerode、Mealy、观测完备性与强保持理论; [13] 为 Linux v6.17 `bpf_loop` API; [20]–[30] 为 RPM/漏洞流及 eBPF 相邻工作, 其中 Rex 为 [28]、`bpfverify` 为 [30]; [31]–[34] 为明确标注为预印本的 2026 工作; [35]–[54] 为经典、后续与案例索引中的 `weird-machine` 原始论文或第一手技术报告。